

Roady 빌드·배포 가이드

GitLab 저장소를 클론한 뒤 Docker Compose로 프론트엔드, Spring 백엔드, 서버 AI, MySQL, Redis, MQTT를 함께 실행하는 절차를 정리한다.

1. 구성 및 버전

구분	제품/버전	설정 및 비고
JVM	Eclipse Temurin JDK/JRE 21	백엔드 Dockerfile의 빌드·실행 이미지
WAS	Spring Boot 4.0.7 내장 Servlet 컨테이너	별도 WAS 설치 없이 <code>app.jar</code> 실행
빌드 도구	Gradle Wrapper 9.5.1	<code>Roady_BE/gradlew</code> 또는 <code>gradlew.bat</code> 사용
백엔드	Spring Boot 4.0.7, MyBatis Spring Boot 4.0.1	포트 8080(컨테이너 내부)
웹 서버	Nginx 1.27-alpine	정적 Vue 파일 제공, <code>/api/ws</code> 를 backend로 reverse proxy
프론트엔드 빌드	Node 24-alpine, npm	<code>package.json</code> 엔진은 Node 22.18 이상 또는 24.12 이상
프론트엔드	Vue 3.5, Vite 8, TypeScript 6	Docker 빌드 결과를 Nginx가 제공
AI 런타임	Python 3.11-slim-bookworm	FastAPI/Uvicorn, Ultralytics, OpenCV
AI 추론	PyTorch 2.8.0, TorchVision 0.23.0	기본 Docker Compose는 CPU, GPU overlay는 CUDA 12.8 wheel
로봇 운영체제	NVIDIA Jetson Orin Nano, JetPack, ROS 2 Humble	로봇에서 Edge AI와 주행 제어 실행
로봇 빌드	colcon, ament_python	<code>Roady_Embedded</code> ROS 2 워크스페이스 빌드
로봇 AI 런타임	CUDA 12 계열, TensorRT FP16	Jetson 장치에서 ONNX를 TensorRT engine으로 변환
관계형 DB	MySQL 8.4	문자셋 <code>utf8mb4</code> , 콜레이션 <code>utf8mb4_unicode_ci</code>
캐시/큐	Redis 7.4-alpine	AI 분석 큐·refresh session·로봇 최신 위치
메시지 브로커	Eclipse Mosquitto 2	MQTT 1883, 로봇 telemetry/command
IDE	저장소에 버전 고정 없음	IDE 없이 Docker/Gradle/npm 명령으로 빌드 가능. 백엔드는 IntelliJ IDEA, 프론트엔드는 VS Code 계열을 권장

IDE 버전과 운영체제 버전은 프로젝트 파일에 고정되어 있지 않다. 재현성은 IDE가 아니라 Docker 이미지, Gradle Wrapper, `package-lock.json`을 기준으로 확보한다.

2. 소스 클론 및 사전 조건

2.1 필요한 도구

- Git
- Docker Engine 또는 Docker Desktop
- Docker Compose v2 (`docker compose` 명령)
- GPU 추론을 사용할 경우 NVIDIA GPU, NVIDIA Driver, NVIDIA Container Toolkit

```
git clone <GitLab 저장소 주소>
cd S15P11A404
docker --version
docker compose version
```

2.2 환경 파일 준비

실제 비밀 값은 Git에 커밋하지 않는다. 백엔드 컨테이너가 읽는 파일은 `Roady_BE/.env`이며, 시작점은 `Roady_BE/.env.example`이다.

```
cp Roady_BE/.env.example Roady_BE/.env
```

Windows PowerShell:

```
Copy-Item Roady_BE/.env.example Roady_BE/.env
```

`MYSQL_DATABASE`, `SPRING_DATASOURCE_URL`의 DB 이름, `MYSQL_USER`, `SPRING_DATASOURCE_USERNAME`을 동일하게 맞춘다. 현재 Compose 기본 DB 이름은 `roadbuddy`다. 운영에서는 반드시 임의의 강한 비밀번호와 JWT secret으로 교체한다.

Compose 파일의 `${...}` 치환값은 저장소 루트의 `.env`, 셸 환경 변수 또는 명령행에서 공급한다. `Roady_BE/.env`의 `env_file` 값은 컨테이너 내부 환경에 주입되지만 Compose 파일의 `${...}` 치환값을 자동으로 공급하지 않는다.

3. Docker Compose 빌드 및 실행

3.1 전체 서비스 실행

```
docker compose up -d --build
docker compose ps
```

서비스와 포트는 다음과 같다.

서비스	컨테이너 포트	호스트 기본 포트	확인 방법
frontend	80	3000	<code>http://localhost:3000</code>
backend	8080	8081	<code>http://localhost:8081/actuator</code> 는 별도 노출하지 않음. 프론트 <code>/api</code> proxy 사용
ai	8000	8000	<code>http://localhost:8000/health/live</code>
mysql	3306	3306	MySQL client 또는 healthcheck
redis	6379	6379	<code>redis-cli ping</code>
mqtt	1883	1883	Mosquitto client

프론트엔드에서는 `http://localhost:3000/api/...`를 호출한다. Nginx가 `/api/`를 `http://backend:8080/api/`로 전달하므로 브라우저에서 `localhost:8081`을 API base URL로 직접 지정할 필요가 없다.

3.2 빌드 로그 및 상태 확인

```
docker compose logs -f ai
docker compose logs -f backend
docker compose logs -f frontend
docker compose ps
```

AI 컨테이너는 모델 해시 검증과 warm-up 후 `/health/ready`가 성공한다. backend는 AI healthcheck가 성공한 뒤 시작한다.

4. 환경 변수

4.1 MySQL, Spring, 공통 환경 (`Roady_BE/.env`)

변수	예시/기본값	설명
MYSQL_DATABASE	roady	MySQL 초기 데이터베이스
MYSQL_USER	별도 지정	애플리케이션 DB 계정
MYSQL_PASSWORD	별도 지정	애플리케이션 DB 비밀번호
MYSQL_ROOT_PASSWORD	별도 지정	MySQL root 비밀번호
SPRING_DATASOURCE_URL	jdbc:mysql://mysql:3306/roady?...	backend 컨테이너 기준 DB 주소
SPRING_DATASOURCE_USERNAME/PASSWORD	DB 계정과 동일	Spring JDBC 접속 정보
SPRING_DATA_REDIS_HOST/PORT	redis / 6379	Redis 주소
ROADY_DEFAULT_USERS_ENABLED	true	admin/inspector/repairer/viewer 샘플 계정 생성
ROADY_SAMPLE_DATA_ENABLED	true	로봇, 파손 시연 데이터 생성
MQTT_ENABLED	true	MQTT adapter 활성화
MQTT_BROKER_IP/PORT	mqtt / 1883	Mosquitto 주소
ROADY_ROBOT_LOCATION_REDIS_KEY_PATTERN	roady:robots:{robotId}:location	최신 위치 Redis key
ROADY_ROBOT_LOCATION_TTL	5m	최신 위치 보관 시간
ROADY_ROBOT_LOCATION_RDB_PERSISTENCE_INTERVAL	1m	위치 RDB 저장 주기
ROADY_AI_IMAGE_ANALYSIS_ENDPOINT_URL	http://ai:8000/analyze	Spring → AI endpoint
ROADY_AI_IMAGE_ANALYSIS_WORKER_ENABLED	true(Compose)	Redis AI worker 활성화
ROADY_AI_IMAGE_ANALYSIS_QUEUE_KEY	roady:damage:image-analysis:queue	분석 대기 큐
ROADY_AI_IMAGE_ANALYSIS_DEAD_LETTER_QUEUE_KEY	roady:damage:image-analysis:dead-letter	실패 큐
ROADY_AI_IMAGE_ANALYSIS_BATCH_SIZE	3	worker polling batch
ROADY_AI_IMAGE_ANALYSIS_POLL_DELAY_MS	1000	polling 간격
ROADY_AI_IMAGE_ANALYSIS_REQUEST_TIMEOUT	3m	AI 요청 timeout
ROADY_KAKAO_GEOCODING_ENABLED	false	Kakao 역지오코딩 사용 여부
ROADY_KAKAO_REST_API_KEY	비밀 값	Kakao Local REST API key
ROADY_CORS_ALLOWED_ORIGINS	프론트 origin 목록	선표로 구분

JWT secret은 roady.jwt.secret으로 관리한다. 운영에서는 application.properties의 개발용 기본값을 사용하지 말고 Spring 외부 설정 (ROADY_JWT_SECRET 또는 실행 인자)으로 교체한다. access token 기본 유효기간은 30분, refresh token은 14일이다.

4.2 Compose, 프론트, AI 환경

변수	기본값	설명
FRONTEND_PORT	3000	frontend 호스트 포트
VITE_API_BASE_URL	/api	빌드 시 프론트 API base URL
AI_PORT	8000	AI 호스트 포트
ROADY_AI_DEVICE	cpu(Compose)	cpu 또는 GPU device 번호
ROADY_AI_IMGSZ	1024(Compose)	추론 입력 크기
ROADY_AI_INFERENCE_CONCURRENCY	1	한 번에 처리할 추론 수
ROADY_AI_VERIFY_MODEL_HASH	true	weight SHA-256 검증

변수	기본값	설명
ROADY_AI_WARMUP_ENABLED	true	readiness 전 warm-up
ROADY_AI_MODEL_PATH	서버 v4 weight 경로	모델 파일 경로
ROADY_AI_MODEL_SHA256_PATH	.sha256 경로	모델 해시 파일
ROADY_AI_MAX_IMAGES	50	한 요청 이미지 개수
ROADY_AI_MAX_IMAGE_BYTES	20MB	이미지 1개 제한
ROADY_AI_MAX_REQUEST_BYTES	200MB	전체 요청 제한
ROADY_AI_SCORE_MINOR_MAX_PIXELS	10000	score 구간 1 상한
ROADY_AI_SCORE_MODERATE_MAX_PIXELS	50000	score 구간 2 상한
ROADY_AI_SCORE_MAX_PIXELS	300000	score 100 도달 픽셀 수

5. 주요 설정, 계정 파일 목록

파일	용도
docker-compose.yml	전체 서비스, 포트, 네트워크, volume, healthcheck
docker-compose.gpu.yml	NVIDIA GPU overlay
Roady_BE/.env	운영/로컬 비밀값. Git 커밋 금지
Roady_BE/.env.example	환경 변수 템플릿
Roady_BE/src/main/resources/application.properties	Spring 기본 설정 및 환경 변수 매핑
Roady_BE/src/main/resources/application-local.properties	로컬 profile 설정
Roady_BE/src/main/resources/rdb/schema.sql	애플리케이션 시작 시 초기화되는 최신 DB schema
Roady_BE/Dockerfile	Gradle build 및 Temurin JRE 실행 이미지
Roady_FE/Dockerfile	Node build 및 Nginx 실행 이미지
Roady_FE/nginx.conf	/api, /ws, SPA fallback reverse proxy
ROADY_AI/Dockerfile	Python-PyTorch-모델을 포함한 AI 이미지
ROADY_AI/models/server/*.sha256	서버 AI weight 무결성 검증
Roady_BE/docker/mosquitto/mosquitto.conf	MQTT listener/persistence 설정

샘플 계정은 ROADY_DEFAULT_USERS_ENABLED=true일 때 생성된다. 계정은 admin, inspector, repairer, viewer이며 초기 샘플 비밀번호는 애플리케이션 initializer에 정의된 개발용 값이다. 운영 환경에서는 seed를 끄고 별도 계정을 생성한다.

6. 배포 특이사항 및 운영 점검

- 1. MySQL, Redis, Mosquitto, AI가 healthy가 된 뒤 backend가 시작되므로 첫 기동은 이미지 다운로드와 AI warm-up 시간만큼 기다린다.
- 2. docker compose down은 named volume을 보존한다. DB까지 삭제하려면 down -v가 필요하지만 데이터가 삭제되므로 운영에서는 실행하지 않는다.
- 3. mysql-data, redis-data, mqtt-data는 영속 volume이다. EC2 장애에 대비해 별도 백업을 보관한다.
- 4. 프론트 API 주소는 운영 도메인의 /api를 사용한다. localhost:3000/api 요청이 Nginx를 거쳐 backend로 전달되어야 한다.
- 5. 운영 도메인이 바뀌면 ROADY_CORS_ALLOWED_ORIGINS와 TLS reverse proxy 설정을 함께 변경한다.
- 6. Mosquitto 기본 설정은 allow_anonymous true인 시연용이다. 운영에서는 인증-ACL-TLS(MQTTS)를 적용한다.
- 7. AI 모델 파일과 .sha256 파일을 함께 배포한다. 해시가 맞지 않으면 readiness가 실패한다.
- 8. 운영에서는 JWT secret, DB 비밀번호, Kakao API key를 Secret Manager 또는 EC2 환경 변수로 주입한다.
- 9. /health/live, /health/ready, /model-info는 운영 모니터링에 사용하되 외부 공개 범위를 제한한다.

7. Jetson 로봇 빌드 및 실행

Roady_Embedded는 NVIDIA Jetson Orin Nano에서 실행되는 ROS 2 워크스페이스다. 점자블록 추종과 파손 탐지, 장애물 감지, 모터 제어, 위치·텔레메트리 전송 및 파손 이미지 자동 업로드 기능을 제공한다. Embedded 모듈은 JVM과 별도 Web Server/WAS를 사용하지 않는다.

7.1 실행 환경 확인

항목	환경
대상 장치	NVIDIA Jetson Orin Nano
Python	Python 3
ROS	ROS 2 Humble
CUDA	CUDA 12 계열
AI Runtime	TensorRT
빌드 방식	colcon, ament_python

배포 Jetson의 실제 버전은 장치에서 다음 명령으로 확인한다.

```
cat /etc/nv_tegra_release
dpkg-query -W nvidia-jetpack 2>/dev/null || true
nvcc --version
dpkg-query -W 'libnvinfer*' 2>/dev/null
python3 --version
```

7.2 프로젝트 구성 및 저장소 준비

```
Roady_Embedded/
├─ src/
│  ├─ control/      # 점자블록 추종, 모터·주행 제어, launch
│  ├─ hardware/     # 카메라, GPS, GPIO 및 센서 인터페이스
│  ├─ perception/   # 파손·장애물 AI 추론과 후처리
│  ├─ communication/ # HTTP 이미지 업로드, MQTT 텔레메트리
│  └─ storage/      # 파손 이벤트와 이미지 로컬 저장
├─ artifacts/       # ONNX 및 TensorRT engine
├─ config/          # Fast DDS 설정
└─ data/            # 런타임 파손 데이터
```

```
git clone https://lab.ssafy.com/s15-webmobile3-sub1/S15P11A404.git
cd S15P11A404/Roady_Embedded
```

기존 저장소를 갱신할 때는 저장소 루트에서 pull한 뒤 Embedded 디렉터리로 이동한다.

```
cd ~/S15P11A404
git pull
cd Roady_Embedded
```

7.3 Edge AI 모델 생성

현재 Jetson에서 TensorRT engine이 정상 로딩되면 다시 생성할 필요가 없다. 모델, JetPack, CUDA 또는 TensorRT 버전이 변경되었거나 engine 역직렬화 오류가 발생하면 해당 Jetson에서 다시 생성한다.

프로젝트 루트에서 점자블록 파손 모델을 ONNX로 export한다.

```
cd ~/S15P11A404

yolo export \
  model=ROADY_AI/models/edge/tactile_damage_candidate_yolo26n_best.pt \
  format=onnx \
  imgsz=768 \
  batch=1 \
  dynamic=False \
  simplify=False \
  opset=17 \
  device=cpu \
  end2end=False

mkdir -p Roady_Embedded/artifacts/tactile_damage_candidate/v4
cp ROADY_AI/models/edge/tactile_damage_candidate_yolo26n_best.onnx \
  Roady_Embedded/artifacts/tactile_damage_candidate/v4/
```

배포할 Jetson에서 TensorRT FP16 engine을 생성한다.

```
cd ~/S15P11A404/Roady_Embedded

/usr/src/tensorrt/bin/trtexec \
  --onnx=artifacts/tactile_damage_candidate/v4/tactile_damage_candidate_yolo26n_best.onnx \
  --saveEngine=artifacts/tactile_damage_candidate/v4/tactile_damage_candidate_yolo26n_best.engine \
  --fp16

/usr/src/tensorrt/bin/trtexec \
  --onnx=artifacts/obstacle_lower_limb/best.onnx \
  --saveEngine=artifacts/obstacle_lower_limb/best.engine \
  --fp16
```

기본 모델 경로:

```
artifacts/tactile_damage_candidate/v4/tactile_damage_candidate_yolo26n_best.engine
artifacts/obstacle_lower_limb/best.engine
```

engine 로딩 확인:

```
/usr/src/tensorrt/bin/trtexec \
  --loadEngine=artifacts/tactile_damage_candidate/v4/tactile_damage_candidate_yolo26n_best.engine
```

TensorRT engine은 GPU 아키텍처와 JetPack/CUDA/TensorRT 버전에 영향을 받으므로 환경이 다른 Jetson에서 생성한 파일을 그대로 사용하지 않는 것이 안전하다.

7.4 ROS 및 Python 의존성 설치

```
cd ~/S15P11A404/Roady_Embedded
source /opt/ros/humble/setup.bash

sudo rosdep init 2>/dev/null || true
rosdep update
rosdep install --from-paths src --ignore-src -r -y

python3 -m pip install \
  'paho-mqtt>=1.6,<3' requests \
```

```
adafruit-blinka \
adafruit-circuitpython-servokit \
adafruit-circuitpython-pca9685
```

전체 launch에는 `ydliidar_ros2_driver`가 필요하다.

```
ros2 pkg prefix ydliidar_ros2_driver
```

명령이 실패하면 팀에서 사용하는 YDLidar ROS 2 드라이버를 설치하거나 `src/ydliidar_ros2_driver` 소스를 복원한다. LiDAR를 launch 인자로 비활성화해도 launch 생성 과정에서 패키지 경로를 조회하므로 패키지 자체는 필요하다.

7.5 ROS 워크스페이스 빌드

```
cd ~/S15P11A404/Roady_Embedded
source /opt/ros/humble/setup.bash

PYTHONNOUSERSITE=1 colcon build --symlink-install
source install/setup.bash
```

`PYTHONNOUSERSITE=1`은 사용자 경로의 setuptools와 ROS 2 Humble 시스템 Python 패키지 충돌을 방지한다.

새 터미널마다 다음 환경을 적용한다.

```
cd ~/S15P11A404/Roady_Embedded
source /opt/ros/humble/setup.bash
source install/setup.bash
export FASTRTPS_DEFAULT_PROFILES_FILE="$PWD/config/fastdds_udp_only.xml"
```

Fast DDS UDP 설정은 root GPIO 노드와 일반 사용자 ROS 노드 사이의 shared-memory 통신 문제를 피하기 위해 사용한다. `sudo`로 실행하는 ROS 노드에도 같은 ROS 환경과 환경변수를 적용한다.

7.6 장치 및 권한 확인

카메라

```
ls -l /dev/v4l/by-id/
sudo usermod -aG video "$USER"
```

- 점자 카메라: `usb-046d_Brio_100_2515ZBA0WRC8-video-index0`
- 장애물 카메라: `usb-HBVCAM_Camera_USB_Camera_HB202400001-video-index0`

장애물 카메라는 `obstacle_camera_device_path` launch 인자로 변경할 수 있다. 점자 카메라 장치가 달라지면 `bringup_teleop.launch.py`의 장치 경로를 수정한다.

LiDAR

```
ls -l /dev/ttyUSB* /dev/ttyACM* 2>/dev/null
sudo usermod -aG dialout "$USER"
```

그룹 추가 후 다시 로그인한다. 실제 포트와 baud rate가 `ydliidar_ros2_driver`의 `params/X4-Pro.yaml`과 일치해야 한다.

I2C/PCA9685

```
ls -l /dev/i2c*
sudo i2cdetect -y 1
```

PCA9685 주소와 실제 실행 계정의 I2C 접근 권한을 확인한다.

7.7 안전 시험 및 전체 실행

최초 실행은 바퀴가 지면에 닿지 않도록 로봇을 고정하고 모터, LiDAR 및 서버 업로드를 비활성화한다.

```
ros2 launch control bringup_all.launch.py \
  start_drive:=false \
  enable_lidar:=false \
  upload_enabled:=false
```

점자블록 추종과 파손 탐지만 모터 없이 확인하려면 다음 명령을 사용한다.

```
ros2 launch control bringup_teleop.launch.py start_drive:=false
```

AI 추론도 제외하려면 다음과 같이 실행한다.

```
ros2 launch control bringup_teleop.launch.py \
  start_drive:=false \
  start_damage_detection:=false
```

전체 시스템 실행:

```
ros2 launch control bringup_all.launch.py
```

기본적으로 모터, LiDAR, 파손 탐지, 영상 녹화, 위치, MQTT telemetry 및 서버 업로드가 활성화된다. 서버 업로드 주소를 명시하려면 다음과 같이 실행한다.

```
ros2 launch control bringup_all.launch.py \
  upload_enabled:=true \
  upload_base_url:=https://i15a404.p.ssafy.io
```

미인증 로봇 업로드는 **robotId**로 소유자를 식별하므로 access token을 사용하지 않는다. 업로드 없이 실행하려면 **upload_enabled:=false**를 사용한다.

장치나 모델 경로 변경 예시:

```
ros2 launch control bringup_all.launch.py \
  obstacle_camera_device_path:='<실제 장치명>' \
  damage_model_path:=/absolute/path/to/damage.engine \
  obstacle_model_path:=/absolute/path/to/obstacle.engine
```

모터 안전 수칙:

1. 최초 실행 시 바퀴가 지면에 닿지 않도록 차체를 고정한다.
2. **start_drive:=false**로 카메라, AI, LiDAR와 비상 정지를 먼저 확인한다.
3. 전원 차단 수단을 작업자 가까이에 둔다.

4. 조향, 전진·후진 방향과 PCA9685 채널을 확인한다.
5. 저속·짧은 거리 시험 후 실제 주행 환경으로 이동한다.
6. 사람 주변에서는 `/emergency_stop` 동작을 먼저 검증한다.

```
ros2 node list
ros2 topic list
ros2 topic echo /emergency_stop
ros2 topic echo /damage/events
```

7.8 저장 및 서버 연동

항목	경로 또는 값
로컬 파손 데이터	<code>data/damage_events</code>
미업로드 이벤트	<code>data/damage_events/pending</code>
파손 원본/ROI 이미지	<code>data/damage_events/images</code>
기본 녹화 영상	<code>~/Y%M%d_%H%M%S.mp4</code>
벤치마크 보고서	<code>reports/</code>
장애물 캡처	<code>~/roady_dataset/obstacle_lower_limb</code>
서버 URL	<code>https://i15a404.p.ssafy.io</code>
파손 등록 API	<code>/api/damages</code>
MQTT broker	<code>i15a404.p.ssafy.io:1883</code>

파손 데이터는 `/driving/finished=true` 이후 서버로 자동 업로드된다. 업로드 성공 시 pending JSON과 연결 이미지가 삭제되고 실패 데이터는 재시도를 위해 남는다. Embedded 모듈은 DB에 직접 접속하지 않는다.

상대 저장 경로를 유지하기 위해 launch는 `Roady_Embedded` 루트에서 실행한다. 자동 디스크 정리 기능이 없으므로 다음 명령으로 주기적으로 확인한다.

```
du -sh data/damage_events reports ~/roady_dataset 2>/dev/null
find data/damage_events/pending -type f 2>/dev/null | wc -l
find ~ -maxdepth 1 -type f -name '*.mp4' -printf '%TY-%Tm-%Td %TT %s %p\n' 2>/dev/null
```

미업로드 파일을 확인하지 않고 삭제하면 안 된다. 운영 전 영상 보존 기간, 최대 디스크 사용량, 업로드 실패 데이터 보존 기간 및 로그 회전 주기를 정한다.